

KICKSTARTER

For creators

Log in

Crafts Dance Design Fashion Film Food Games Journalism Music Photography Publishing Technology Theater

JAVA: SOLID Principles and Top Design Patterns



Master the most important and fundamental Design Patterns in modern Software Engineering with JAVA

Follow along!

Created by

P.J. Paweska

7 backers pledged CA\$ 536 to help bring this project to life.

 **Last updated** November 30, 2023

[Campaign](#)[Rewards](#)[Creator](#)[FAQ ⁹](#)[Updates ¹](#)[Comments ¹⁰](#)

JAVA: SOLID Principles and Top Design Patterns

📍 [Toronto, Canada](#) 🛠️ [Software](#)

CA\$ 536

pledged of CA\$ 500 goal

7

backers

Funding period

Oct 31 2023 - Nov 30 2023 (30 days)

Story



WHY DO YOU NEED THIS COURSE?

In **Java**, both **SOLID Principles** and **Design Patterns** play a crucial role in

the creation of professional Software Architecture.

Java, being an Object-Oriented Programming language, aligns seamlessly with these guidelines. SOLID Principles and Design Patterns combined pave the way for producing Java applications that are **Flexible**, **Maintainable**, and **Robust**. A sound understanding of both these aspects can lead to a more fruitful software development career.

Applying the SOLID Principles and Design Patterns in Java (and in software development at large) offers numerous advantages:

- ✓ **Maintainability:** Embracing SOLID principles and Design Patterns produces code that's simpler to maintain, allowing for additions or alterations without the high risk of inducing bugs. This cuts down on maintenance effort and time.
- ✓ **Scalability:** These principles and patterns push for a modular, loosely coupled design, facilitating easier scaling. Adding or extending components without disturbing the system's other sections lets your software evolve smoothly.
- ✓ **Reusability:** Code that is well Architected results in components that can be reused across different parts of a project or even in separate projects.
- ✓ **Testability:** Code built on these architectural principles and patterns is generally easier to test due to the enhanced isolation of components. This results in superior test coverage and sturdier software.
- ✓ **Reduced Complexity:** Using SOLID Principles and Design Patterns prompts a clean, straightforward design, making your code less intricate and more comprehensible.
- ✓ **Collaboration:** Adherence to these concepts simplifies collaboration. Developers can focus on specific components without needing a deep grasp of the entire codebase, leading to improved teamwork.
- ✓ **Code Quality:** SOLID Principles, combined with Design Patterns, bolster code quality, yielding less error-prone, more readable code that follows industry best practices.
- ✓ **Future-Proofing:** Both SOLID Principles and Design Patterns ensure your code remains flexible to future shifts and demands, preventing the complications of "spaghetti code."
- ✓ **Easier Debugging:** Implementing these principles and patterns makes debugging more straightforward. Problems can be narrowed down to

particular components, easing the bug-fixing process.

✓ **Career Advancement:** Proficiency in both SOLID Principles and Design Patterns is a sought-after trait in the software development sector. It indicates your prowess in crafting maintainable, high-caliber software, elevating your job prospects and career progression.



Java, with its strong foothold in the **enterprise world**, demands robust, maintainable, and scalable solutions.

Given Java's inherent Object-Oriented nature, the S.O.L.I.D Principles and Design Patterns align seamlessly, enabling developers to harness the full potential of Java.

This course is not just an academic exercise but a transformative journey, taking Java developers from merely coding to crafting resilient and efficient architectures.

There are a number of key reasons why Java is an influential and widely used programming language:

✓ **Object-Oriented:** Java's object-oriented programming (OOP) features promote modularity, code reusability, and maintainability, making it suitable for building complex and scalable software systems.

✓ **Platform Independence:** Java's "Write Once, Run Anywhere" philosophy allows developers to write code that can run on any platform with a Java Virtual Machine (JVM). This platform independence is crucial for cross-platform compatibility.

✓ **Versatility:** Java is used in a wide range of applications, from mobile app development (Android), web development (Java EE), and desktop

applications to embedded systems, scientific computing, and large-scale enterprise applications.

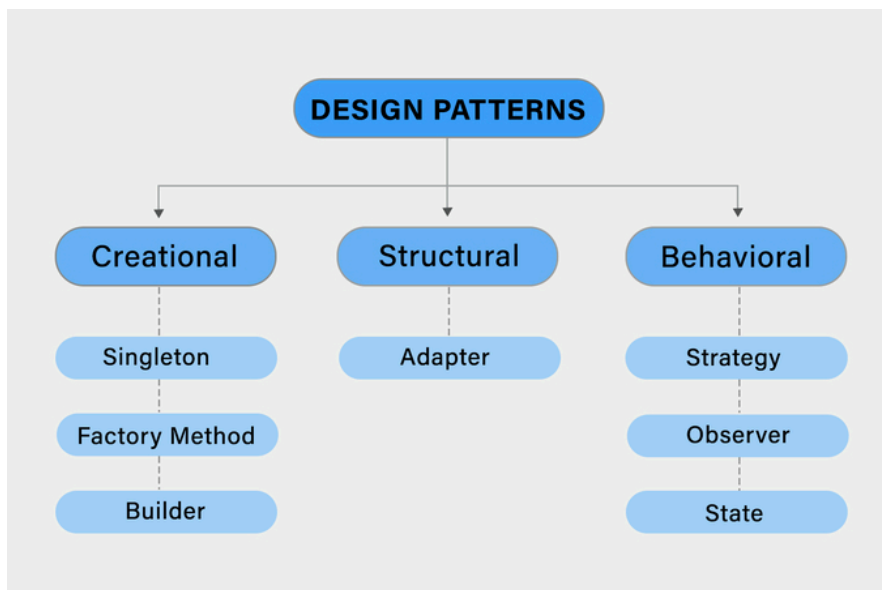
✓ **Strong Ecosystem:** Java has a vast and mature ecosystem of libraries, frameworks, and tools that simplify development, testing, and deployment. This ecosystem includes popular frameworks like Spring, Hibernate, and JavaFX.

✓ **Longevity:** Java has been around for a long time and continues to be relevant in modern software development, which means that developers with Java skills are in demand.

✓ **Job Opportunities:** Java is one of the most widely used programming languages in the job market, offering numerous opportunities for developers and programmers.

The importance of Java stems from its **versatility, platform independence, strong ecosystem**, and the ability to build **secure, robust**, and **scalable** software. It remains a significant player in various domains, making it a valuable language for developers and businesses.

MASTER THESE FUNDAMENTAL PATTERNS



Competitive Advantage

Mastering these concepts is a game-changer for Java developers. It not only provides a competitive edge in the saturated job market but also equips them for advanced roles, ensuring their code remains intuitive, modular, and in sync with Java's evolving features. Furthermore, a shared understanding among teams enhances collaboration, making code reviews more streamlined and effective.

WHY IS S.O.L.I.D IMPORTANT?

The SOLID principles are five fundamental guidelines that ensure our software is easy to maintain, understand, and scale.

S.O.L.I.D equips developers with a foundation that emphasizes clarity, scalability, and maintainability.

A system built on SOLID principles is not just a codebase, but an adaptable, efficient, and robust ecosystem.

WHY ARE DESIGN PATTERNS IMPORTANT?

Design patterns are like tried-and-true templates to recurring software challenges. Here's why they're indispensable:

Standard Solutions: With design patterns, you're not reinventing the wheel. These are established solutions to common problems, saving time and ensuring effective problem-solving.

Code Reusability & Efficiency: Patterns provide a blueprint, making it easier to replicate solutions across different parts of the application or even in different projects, leading to consistency and speed in development.

Improved Communication: Design patterns create a common vocabulary for developers. Instead of explaining a complex design in detail, a

developer can simply refer to the pattern's name, facilitating more efficient and clear communication.

Flexibility & Adaptability: Patterns provide guidelines, not hard-set rules. They can be adapted to fit specific needs, offering flexibility while maintaining the essence of the solution.

Refactoring Friendly: Recognizing parts of your application that can be improved with a design pattern makes refactoring more intuitive. Patterns provide a clear path to better and more structured code.

Together, SOLID principles and design patterns create a synergistic force. While SOLID lays the foundational ethos of how software should be crafted, design patterns offer the tools to bring this vision to life.

A developer who masters both is not just coding; they are crafting art. Their systems are not just functional but are robust, scalable, and maintainable masterpieces.

WHAT YOU WILL LEARN

Embark on a transformative journey where you won't just code better, but you'll Think, Design, Architect, and Innovate better.

Dive deep into the most pivotal design patterns and gain the prowess of seasoned senior developers by grasping the nuances of S.O.L.I.D principles.

SOLID Principles:

✓ **Single Responsibility Principle (SRP):** You'll learn how to design classes and components that have a single, well-defined responsibility, making your code more maintainable and less prone to bugs.

✓ **Open-Closed Principle (OCP):** This principle focuses on designing code that is open for extension but closed for modification. You'll learn techniques for creating flexible and extensible software.

✓ **Substitution Principle (LSP):** You'll understand how to design classes and interfaces that can be used interchangeably (substituted) without

affecting the correctness of the program.

✓ **Interface Segregation Principle (ISP):** You'll learn the importance of defining small, focused interfaces to prevent clients from being forced to depend on methods they do not use.

✓ **Dependency Inversion Principle (DIP):** This principle emphasizes decoupling high-level modules from low-level modules and relying on abstractions. You'll learn how to achieve this decoupling through **dependency injection** and **inversion of control**.

Design Patterns:

✓ **Creational Patterns:** Patterns like Singleton, Factory Method, Abstract Factory, Builder, and Prototype, provide solutions for object creation and instantiation.

✓ **Structural Patterns:** These patterns, including Adapter, Bridge, Composite, Decorator, Facade, and Proxy, help you organize and structure classes and objects to form larger and more complex structures.

✓ **Behavioral Patterns:** This family of design patterns such as Observer, Strategy, Command, Chain of Responsibility, and State, address how objects interact and communicate with each other.

✓ **Architectural Patterns:** We may also delve into architectural patterns like Model-View-Controller (MVC) and Model-View-ViewModel (MVVM), which define the overall structure and organization of an application.

✓ **Concurrency Patterns:** These patterns, like Thread Pool, Producer-Consumer, and Active Object, focus on managing concurrent and parallel execution in multi-threaded applications.

✓ **Anti-Patterns:** You'll gain an understanding of common design pitfalls and anti-patterns to avoid in your software development process.

✓ **Real-World Application:** You'll learn how to apply these patterns in real-world scenarios to solve common software design challenges and improve the maintainability, flexibility, and scalability of your Java applications.

The Need: Great developers transcend coding. They craft architectures, devise patterns, and tackle complex systems with elegance. To bear the titles "Software Engineer" or "Software Architect", one must possess a treasure trove of design patterns.

The Solution: Our curated journey ventures beyond mere coding, emphasizing understanding and application. We unveil the **top**

fundamental GoF Design Patterns, connecting them seamlessly with the **S.O.L.I.D Design Principles**. And the cherry on top? Experience it all within the exciting world of **Java Technology Stack**.

The Challenge: Use what you have learned by architecting a solution to the famous John Conway's **Game of Life simulation**, bringing it alive on your screen.

Why Patterns & Architecture Matter: Architectures are the blueprints of software, much like those for skyscrapers. Just as buildings have universal, time-tested components, so does software. These are the almighty "Design Patterns". Well-deployed, they remove redundancy, simplify complexity, and ensure that every developer on your team is focused on their given task with clarity and purpose.

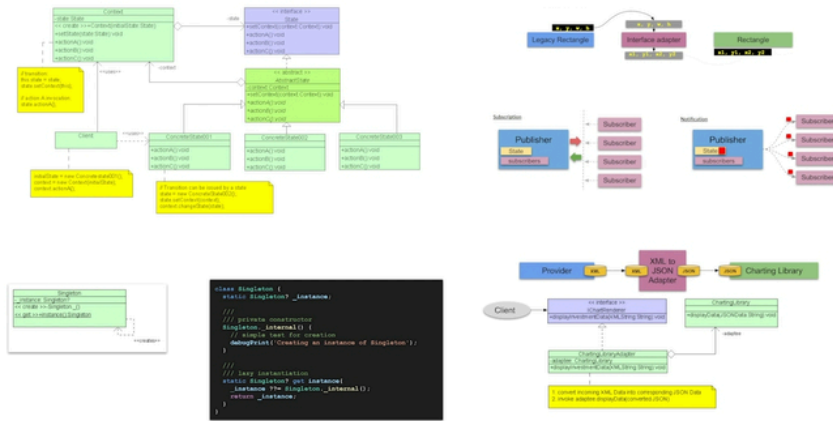
Our Approach: Beyond coding, this course will teach you about the conceptual power of design patterns. We'll not only teach you the patterns but enlighten you on **when**, **why**, and **how** to adapt and modify them. And in culmination, we'll integrate these patterns into your final project giving you an overall sense of how they all fit together.

Beyond Skills

This is more than just skills; it's a paradigm shift. Students will learn about optimizing every line of code, making informed architectural decisions, and setting a gold standard in Java development.

Through SOLID and design patterns, developers will find themselves better equipped to navigate the dynamic landscape of modern software engineering.

COURSE HIGHLIGHTS



Grasp Fundamental Design Patterns: Delve deep into the WHAT, WHY, and HOW of design patterns that underpin modern software engineering.

S.O.L.I.D Foundation: Understand and apply the S.O.L.I.D principles as seasoned developers do, ensuring robustness and scalability in your software designs.

Architectural Exposition: Benefit from *detailed UML expositions* of each pattern. Visualize the architecture before diving into the code.

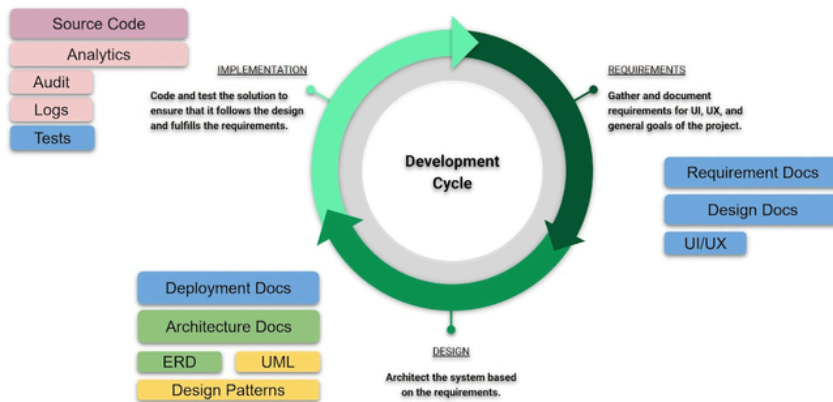
Real-World Scenarios & Hands-On Coding: Witness the patterns in action as they solve real-world problems. Code along and internalize the implementations.

Strategic Application: Learn how to discern the best pattern for varied architectural challenges, amplifying the quality and efficiency of your solutions.

Comprehensive Assignments: Strengthen your grasp by working on Java-based assignments tailored to reinforce pattern comprehension.

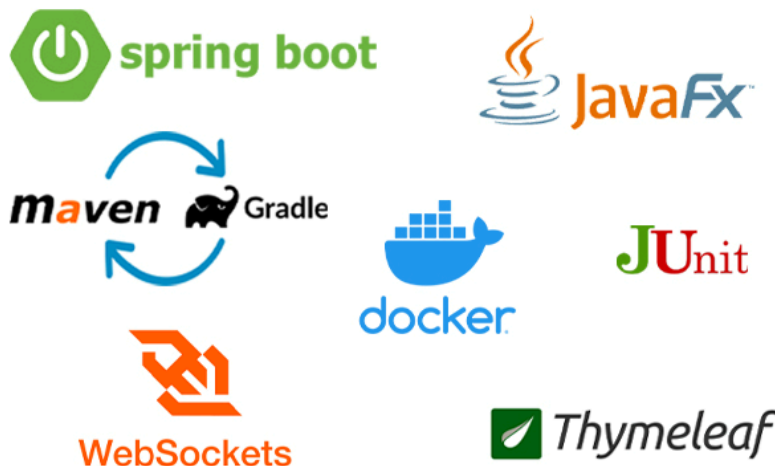
STRATEGIES FOR EFFECTIVE IMPLEMENTATION

Good development habits start with **good organization**. We will teach you how to break down the development process into a much more **manageable** and **compartmentalized development cycle**.



When you **organize** your approach to development, **utilize S.O.L.I.D Principles**, and **incorporate Design Patterns**, you will be able to tackle **more complex projects**, but most importantly, **without increasing the complexity** of your approach.

TECHNOLOGY HIGHLIGHTS



- **Spring Boot**: As a foundational framework, Spring Boot makes it easy to create stand-alone, production-grade Spring-based applications. It provides embedded Tomcat or Jetty servers, simplifying web app development.
- **JavaFX**: When using Java for the front-end, JavaFX provides a rich set of graphics and UI controls for developing rich client applications.

Although JavaFX is typically used for desktop applications, with tools like JPro, we could run JavaFX applications in a web browser.

- **Thymeleaf:** We will also use traditional web technologies for the frontend (HTML, CSS, JavaScript). Since Thymeleaf is integrated with Spring Boot it is a great choice since Thymeleaf is a natural templating engine that can be used to create dynamic web pages using Spring data.
- **Spring WebSockets:** For a more interactive experience where the state of the game updates in real-time, WebSockets with Spring can be utilized. It allows for a two-way interactive communication session between the user's browser and the server.
- **JUnit:** For unit testing the simulation logic to ensure the rules of the Game of Life are correctly implemented.
- **Maven/Gradle:** To manage the project's build cycle, dependencies, and packaging.
- **Docker:** Once you've developed your application, Docker can be used to containerize it, ensuring that it runs consistently across different environments. This is especially useful if you plan on deploying your solutions to the cloud or sharing it with others.

Who Is This Course For?

JAVA Developers: Those hungry to design, architect, and produce superior code, elevating their software craftsmanship.

Architecture Enthusiasts: Developers aiming to delve deep into the essence of top-tier software architecture, deriving beauty from well-structured systems.

The Promise:

By journey's end, you won't just be coding; **you'll be architecting**. With time-tested **design patterns at your fingertips and architectural wisdom in your arsenal**, you'll craft systems where every line of code is a testament to elegance and efficiency of your vision and skill.

Support Our Mission:

Join us on this Kickstarter journey. Together, we'll reshape the narrative of software development, forging a community of adept architects and sublime software engineers. Your support will bring this holistic learning experience to life, heralding a new era of superior software design and architecture.

Testimonials

**Sandra Z.**

★★★★★ 2 days ago



The course was not just informative but also incredibly engaging and challenging, making it an ideal choice for anyone looking to dive into game development with Flutter Flame.

What I loved most about this course was the instructor's approach to teaching. They didn't just walk us through the basics but also provided a series of thought-provoking challenges that pushed us to improve our code and problem-solving skills. These challenges were the real gems of the course, as they allowed me to apply what I learned in practical scenarios and develop a deeper understanding of game development.

Furthermore, the instructor's explanations of game planning were clear and easy to follow. They broke down complex concepts into simple, digestible bits, making it accessible for learners of all levels. The course structure was well-organized, and I always felt like I was building on my knowledge with each new module.

One thing that sets this course apart is the emphasis on not just creating games but creating them efficiently and with best practices in mind.

**Michael L.**

★★★★★ 2 months ago



Amazing course. I recommend it to everyone and encourage Piotr to develop new courses. Piotr was extremely detailed, quick and thorough in answering my questions, some of which were not even on the subject of the lectures, but just on Python.

**Jason D.**

★★★★★ a month ago



solid course. Exceptional production quality. Starts with quick python stuff you need to understand the rest, such as how to create abstract classes in python and enforce interface constraints... Lovely slides and animation as well as UML diagrams (class diagrams and sequence diagrams) for every design pattern and example of design pattern. Talks about pros/cons of each design pattern, link its to similar design patterns not in the course and guidelines for when to use/not use and how the pattern supports solid principles. Nice quick interesting excercises to complete using udemy IDE to ensure you have practical experience. Also suggests some extra bonus excercises you can complete and one final capstone project where you get to use all the design patterns to design and implement a simple game - game of life. Only the coding excercises via ide are necessary to complete course to get certification. May seem like a short course but is jam packed whilst really easy to understand. Slides are lovely so dont have to make to many notes and can focus on writing code. Worth every penny.

**Shukhrat K.**

★★★★★ a month ago



This is hands down the best Dart/Flutter course in Udemy. Fantastic lectures, loads of explanations, and a huge number of practical tasks. The final practical task is immensely satisfying and really puts your skills to the test. Highly recommended.

**Udemy User**

★★★★★ 3 weeks ago



Incredible! Wonderful integration of SOLID principle and the design patterns! One of the best courses I've learned.

**Adam Z.**

★★★★★ a month ago



This course is comprehensible, straight to the point, no spare talking. I like pygame examples since I use that library often now. I like Piotr's teaching approach.

Risks and challenges



The subject matter of this course – delving deep into design patterns, SOLID principles, and Java frameworks – is inherently complex. But with our prior experience in delivering successful courses, we have honed our curriculum design process. Our course structure is meticulously crafted, ensuring step-by-step progression, making complex ideas accessible.



Of course, unforeseen circumstances could potentially delay course launch or the release of certain modules. But, with our track record of creating and delivering courses, we have a well-defined content creation pipeline. We've also factored in buffer times to account for any unforeseen hiccups.

[Learn about accountability on Kickstarter](#)

Questions about this project? [Check out the FAQ](#)

[Report this project to Kickstarter](#)

Similar projects to check out

[See more](#)



Save our oceans. Take Wildtype mainstream.

Justin Kolbeck



Zen Tea Brew Infusions: Discover Delicious Rare Shizuoka Tea



Faire in the Field: Year Two!

Patrick Munroe-Smith





Laughing Gull Chocolates Take Flight

Lindsay

10 days left • 81% funded



- Discover

Arts

Comics & Illustration

Design & Tech

Film

Food & Craft

Games

Music

Publishing
- About

About us

Our charter

Press

Jobs
- Support

Help Center

Our Rules

Creator Resources

Forward Funds

Brand assets
- More

Newsletters

Kickstarter Project Updates

The Creative Independent

Mobile apps

Research

KICKSTARTER

English

\$ US Dollar (USD)

